

Below is a **complete practical reference** showing how **1-D, 2-D, and 3-D arrays can be converted into Java 1.8 Collection classes** with example code.

Java collections belong mainly to:

- **List**
- **Set**
- **Queue / Deque**
- **Map**

Common implementations used in Java 8:

- ArrayList
- LinkedList
- Vector
- Stack
- HashSet
- LinkedHashSet
- TreeSet
- PriorityQueue
- ArrayDeque
- HashMap
- LinkedHashMap
- TreeMap
- ConcurrentHashMap

1. Convert 1D Array → Java Collections

Example Array

```
int[] arr = {10, 20, 30, 40, 50};
```

1.1 Array → ArrayList

```
import java.util.*;

public class ArrayToArrayList {
    public static void main(String[] args) {

        Integer[] arr = {10,20,30,40};

        List<Integer> list = new ArrayList<>(Arrays.asList(arr));

        System.out.println(list);
    }
}
```

1.2 Array → LinkedList

```
LinkedList<Integer> list =  
    new LinkedList<>(Arrays.asList(arr));
```

1.3 Array → Vector

```
Vector<Integer> vector =  
    new Vector<>(Arrays.asList(arr));
```

1.4 Array → Stack

```
Stack<Integer> stack = new Stack<>();  
Collections.addAll(stack, arr);
```

1.5 Array → HashSet

```
Set<Integer> set =  
    new HashSet<>(Arrays.asList(arr));
```

1.6 Array → LinkedHashSet

```
Set<Integer> set =  
    new LinkedHashSet<>(Arrays.asList(arr));
```

1.7 Array → TreeSet

```
Set<Integer> set =  
    new TreeSet<>(Arrays.asList(arr));
```

1.8 Array → Queue

```
Queue<Integer> queue =  
    new PriorityQueue<>(Arrays.asList(arr));
```

1.9 Array → Deque

```
Deque<Integer> deque =  
    new ArrayDeque<>(Arrays.asList(arr));
```

1.10 Array → Map

Convert array index → value.

```
Map<Integer,Integer> map = new HashMap<>();

for(int i=0;i<arr.length;i++)
{
    map.put(i, arr[i]);
}

System.out.println(map);
```

2. Convert 2D Array → Java Collections

Example

```
int[][] arr = {
    {1,2,3},
    {4,5,6},
    {7,8,9}
};
```

2.1 2D Array → List

```
List<List<Integer>> list = new ArrayList<>();

for(int[] row : arr)
{
    List<Integer> temp = new ArrayList<>();

    for(int val : row)
    {
        temp.add(val);
    }

    list.add(temp);
}

System.out.println(list);
```

2.2 2D Array → Set

```
Set<Set<Integer>> set = new HashSet<>();
```

```
for(int[] row : arr)
{
    Set<Integer> temp = new HashSet<>();

    for(int val : row)
    {
        temp.add(val);
    }

    set.add(temp);
}
```

2.3 2D Array → Queue

```
Queue<List<Integer>> queue = new LinkedList<>();

for(int[] row : arr)
{
    List<Integer> list = new ArrayList<>();

    for(int val : row)
    {
        list.add(val);
    }

    queue.add(list);
}
```

2.4 2D Array → Map<Row,List>

```
Map<Integer,List<Integer>> map = new HashMap<>();

for(int i=0;i<arr.length;i++)
{
    List<Integer> row = new ArrayList<>();

    for(int val : arr[i])
    {
        row.add(val);
    }

    map.put(i,row);
}
```

3. Convert 3D Array → Java Collections

Example

```
int[][][] arr = {
    {
        {1,2},
        {3,4}
    },
    {
        {5,6},
        {7,8}
    }
};
```

3.1 3D Array → List<List>

```
List<List<List<Integer>>> list3D = new ArrayList<>();

for(int[][] matrix : arr)
{
    List<List<Integer>> list2D = new ArrayList<>();

    for(int[] row : matrix)
    {
        List<Integer> list1D = new ArrayList<>();

        for(int val : row)
        {
            list1D.add(val);
        }

        list2D.add(list1D);
    }

    list3D.add(list2D);
}

System.out.println(list3D);
```

3.2 3D Array → Map<Integer,List>

```
Map<Integer,List<List<Integer>>> map = new HashMap<>();

for(int i=0;i<arr.length;i++)
{
    List<List<Integer>> matrix = new ArrayList<>();

    for(int[] row : arr[i])
```

```

{
    List<Integer> list = new ArrayList<>();

    for(int val : row)
    {
        list.add(val);
    }

    matrix.add(list);
}

map.put(i,matrix);
}

```

4. Java 8 Stream Way (Modern Approach)

Array → List

```

Integer[] arr = {10,20,30,40};

List<Integer> list =
    Arrays.stream(arr)
        .collect(Collectors.toList());

```

Array → Set

```

Set<Integer> set =
    Arrays.stream(arr)
        .collect(Collectors.toSet());

```

Array → Map

```

Map<Integer,Integer> map =
    IntStream.range(0, arr.length)
        .boxed()
        .collect(Collectors.toMap(
            i -> i,
            i -> arr[i]
        ));

```

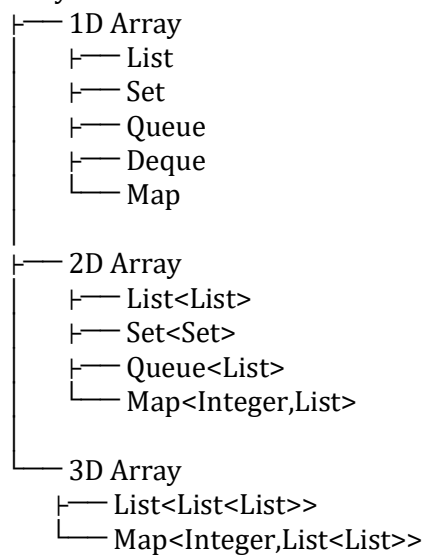
5. Important Collection Methods Used

Collection	Important Methods
List	add(), get(), set(), remove(), size(), contains()

Set	add(), remove(), contains(), size()
Queue	add(), offer(), poll(), peek()
Deque	addFirst(), addLast(), removeFirst(), removeLast()
Map	put(), get(), remove(), containsKey(), keySet()

6. Summary Architecture

Array



If you want, I can also give:

- **Complete Java program converting arrays → ALL Java Collections automatically**
- **Java 8 Stream API version (most asked in interviews)**
- **50 Java coding interview problems on Arrays ↔ Collections conversions.**

Below is a **complete Java 8 program** that converts **1D, 2D, and 3D arrays into almost all major Java Collection classes automatically.**

It demonstrates conversion to:

- **List implementations**
- **Set implementations**
- **Queue / Deque**
- **Map implementations**

This is useful for **Java interviews, Stream API practice, and collection understanding.**

Complete Java Program

Convert Arrays → All Java Collections

```
import java.util.*;
import java.util.stream.*;

public class ArrayToAllCollections {

    public static void main(String[] args) {

        Integer[] array1D = {10,20,30,40,50};

        int[][] array2D = {
            {1,2,3},
            {4,5,6},
            {7,8,9}
        };

        int[][][] array3D = {
            {
                {1,2},
                {3,4}
            },
            {
                {5,6},
                {7,8}
            }
        };

        System.out.println("===== 1D ARRAY CONVERSIONS =====");

        convert1D(array1D);

        System.out.println("\n===== 2D ARRAY CONVERSIONS =====");
```



```

convert2D(array2D);

System.out.println("\n===== 3D ARRAY CONVERSIONS =====");

convert3D(array3D);

}

// 1D ARRAY CONVERSIONS

public static void convert1D(Integer[] arr)
{
    // List Implementations

    List<Integer> arrayList = new ArrayList<>(Arrays.asList(arr));
    List<Integer> linkedList = new LinkedList<>(Arrays.asList(arr));
    List<Integer> vector = new Vector<>(Arrays.asList(arr));

    System.out.println("ArrayList : " + arrayList);
    System.out.println("LinkedList : " + linkedList);
    System.out.println("Vector : " + vector);

    // Stack

    Stack<Integer> stack = new Stack<>();
    Collections.addAll(stack, arr);

    System.out.println("Stack : " + stack);

    // Set Implementations

    Set<Integer> hashSet = new HashSet<>(Arrays.asList(arr));
    Set<Integer> linkedHashSet = new LinkedHashSet<>(Arrays.asList(arr));
    Set<Integer> treeSet = new TreeSet<>(Arrays.asList(arr));

    System.out.println("HashSet : " + hashSet);
    System.out.println("LinkedHashSet : " + linkedHashSet);
    System.out.println("TreeSet : " + treeSet);

    // Queue

    Queue<Integer> priorityQueue = new PriorityQueue<>(Arrays.asList(arr));
    Queue<Integer> linkedQueue = new LinkedList<>(Arrays.asList(arr));

    System.out.println("PriorityQueue : " + priorityQueue);
    System.out.println("LinkedList Queue : " + linkedQueue);

    // Deque

```

```

Deque<Integer> arrayDeque = new ArrayDeque<>(Arrays.asList(arr));

System.out.println("ArrayDeque : " + arrayDeque);

// Map Implementations

Map<Integer,Integer> hashMap = new HashMap<>();
Map<Integer,Integer> linkedHashMap = new LinkedHashMap<>();
Map<Integer,Integer> treeMap = new TreeMap<>();

for(int i=0;i<arr.length;i++)
{
    hashMap.put(i,arr[i]);
    linkedHashMap.put(i,arr[i]);
    treeMap.put(i,arr[i]);
}

System.out.println("HashMap : " + hashMap);
System.out.println("LinkedHashMap : " + linkedHashMap);
System.out.println("TreeMap : " + treeMap);

}

```

// 2D ARRAY CONVERSIONS

```

public static void convert2D(int[][] arr)
{

    List<List<Integer>> list2D = new ArrayList<>();

    for(int[] row : arr)
    {
        List<Integer> list = new ArrayList<>();

        for(int val : row)
        {
            list.add(val);
        }

        list2D.add(list);
    }

    System.out.println("List<List> : " + list2D);

    Set<Set<Integer>> set2D = new HashSet<>();

    for(int[] row : arr)

```

```

{
    Set<Integer> set = new HashSet<>();

    for(int val : row)
    {
        set.add(val);
    }

    set2D.add(set);
}

System.out.println("Set<Set> : " + set2D);

Queue<List<Integer>> queue = new LinkedList<>();

for(int[] row : arr)
{
    List<Integer> list = new ArrayList<>();

    for(int val : row)
    {
        list.add(val);
    }

    queue.add(list);
}

System.out.println("Queue<List> : " + queue);

Map<Integer,List<Integer>> map = new HashMap<>();

for(int i=0;i<arr.length;i++)
{
    List<Integer> list = new ArrayList<>();

    for(int val : arr[i])
    {
        list.add(val);
    }

    map.put(i,list);
}

System.out.println("Map<Row,List> : " + map);
}

```

```
// 3D ARRAY CONVERSIONS
```

```
public static void convert3D(int[][][] arr)
{
    List<List<List<Integer>>> list3D = new ArrayList<>();

    for(int[][] matrix : arr)
    {
        List<List<Integer>> list2D = new ArrayList<>();

        for(int[] row : matrix)
        {
            List<Integer> list1D = new ArrayList<>();

            for(int val : row)
            {
                list1D.add(val);
            }

            list2D.add(list1D);
        }

        list3D.add(list2D);
    }

    System.out.println("List<List<List>> : " + list3D);

    Map<Integer,List<List<Integer>>> map = new HashMap<>();

    for(int i=0;i<arr.length;i++)
    {
        List<List<Integer>> matrix = new ArrayList<>();

        for(int[] row : arr[i])
        {
            List<Integer> list = new ArrayList<>();

            for(int val : row)
            {
                list.add(val);
            }

            matrix.add(list);
        }

        map.put(i,matrix);
    }
}
```

```

        System.out.println("Map<Integer,List<List>> : " + map);
    }
}

```

Example Output

===== 1D ARRAY CONVERSIONS =====

```

ArrayList : [10, 20, 30, 40, 50]
LinkedList : [10, 20, 30, 40, 50]
Vector : [10, 20, 30, 40, 50]
Stack : [10, 20, 30, 40, 50]

```

```

HashSet : [50, 20, 40, 10, 30]
LinkedHashSet : [10, 20, 30, 40, 50]
TreeSet : [10, 20, 30, 40, 50]

```

```

PriorityQueue : [10, 20, 30, 40, 50]
ArrayDeque : [10, 20, 30, 40, 50]

```

```

HashMap : {0=10, 1=20, 2=30, 3=40, 4=50}

```

===== 2D ARRAY CONVERSIONS =====

```

List<List> : [[1,2,3],[4,5,6],[7,8,9]]

```

===== 3D ARRAY CONVERSIONS =====

```

List<List<List>> : [[[1,2],[3,4]],[[5,6],[7,8]]]

```

Collections Covered

Category	Implementations
List	ArrayList, LinkedList, Vector
Stack	Stack
Set	HashSet, LinkedHashSet, TreeSet
Queue	PriorityQueue, LinkedList
Deque	ArrayDeque
Map	HashMap, LinkedHashMap, TreeMap

--	--

If you want, I can also provide a **very advanced interview version**:

- **Single Generic Utility Class that converts ANY array dimension → ANY Java Collection automatically**
- **Using Java 8 Streams + Reflection**
- **Professional reusable library-style code (very impressive in interviews).**

Below are **50 Java coding interview problems focused on Arrays ↔ Collections conversions**. These questions are commonly asked in **Java 8, Stream API, and data structure interviews**.

I organized them into **difficulty levels** so they are easier to study.

50 Java Coding Interview Problems

Arrays ↔ Collections Conversion

No	Problem
1	Convert an array into an ArrayList.
2	Convert an array into a LinkedList.
3	Convert an array into a HashSet.
4	Convert an array into a TreeSet.
5	Convert an array into a LinkedHashSet.
6	Convert an array into a PriorityQueue.
7	Convert an array into a Deque.
8	Convert an array into a Stack.
9	Convert an array into a HashMap where index = key.
10	Convert an array into a LinkedHashMap.

Intermediate Problems

No	Problem
11	Convert an array to a List using Java 8 Streams.
12	Convert a List back to an array.
13	Convert a Set to an array.
14	Convert a Queue to an array.
15	Convert a Map keys into an array.
16	Convert a Map values into an array.
17	Convert an array into a Map using Java Streams.
18	Convert an array into a List without using Arrays.asList().
19	Convert a primitive array (int[]) into List<Integer>.
20	Convert List<Integer> back to int[].

Duplicate & Sorting Problems

No	Problem
21	Convert an array to a Set to remove duplicates.
22	Convert an array to a List and sort it.
23	Convert a List to an array and reverse it.
24	Convert an array to a TreeSet and print sorted elements.
25	Convert a List to a Set and back to List without duplicates.

Map Conversion Problems

No	Problem
26	Convert an array into a Map<String,Integer> with element frequency.

27	Convert an array into a Map where value = square of number.
28	Convert a Map into a List of keys.
29	Convert a Map into a List of values.
30	Convert a Map entry set into an array.

2D Array Conversion Problems

No	Problem
31	Convert a 2D array into List<List<Integer>>.
32	Convert List<List<Integer>> back to a 2D array.
33	Convert a 2D array into Set<Set<Integer>>.
34	Convert a 2D array into Map<Integer,List<Integer>>.
35	Flatten a 2D array into a single List.

3D Array Conversion Problems

No	Problem
36	Convert a 3D array into List<List<List<Integer>>>.
37	Convert List<List<List<Integer>>> back to a 3D array.
38	Flatten a 3D array into a List.
39	Convert a 3D array into Map<Integer,List<List<Integer>>>.
40	Convert nested lists into a multidimensional array.

Advanced Java 8 Stream Problems

No	Problem
41	Convert an array to a List and filter even numbers using Streams.

42	Convert a List to an array and remove duplicates using Streams.
43	Convert an array into a Map using Collectors.toMap().
44	Convert an array into a Set using Streams.
45	Convert a Map into a sorted List.
46	Convert a List into a Map where key = index.
47	Convert a List into a Map where value = length of string.
48	Convert an array into a grouped Map using Streams.
49	Convert a List into a TreeSet using Streams.
50	Convert nested lists into a flat array using flatMap().

Example Interview Problem (Most Asked)

Problem

Convert an array to a list and remove duplicates.

Solution

```
import java.util.*;
import java.util.stream.*;

public class Example {
    public static void main(String[] args) {

        Integer[] arr = {1,2,3,4,3,2,1};

        List<Integer> list =
            Arrays.stream(arr)
                .distinct()
                .collect(Collectors.toList());

        System.out.println(list);
    }
}
```

Output

[1,2,3,4]

Most Important Interview Topics From This List

Focus especially on:

1. `Arrays.asList()`
2. `Collections.addAll()`
3. `List.toArray()`
4. `Stream.collect()`
5. `Collectors.toList()`
6. `Collectors.toSet()`
7. `Collectors.toMap()`
8. `flatMap()`
9. `distinct()`
10. Primitive vs Wrapper conversions

If you want, I can also provide **3 extremely valuable interview resources**:

1 100 Java Stream API Coding Interview Problems

2 Top 50 Java Collection Coding Interview Questions

3 Complete Java Collection Framework Coding Problem Set (200+ Problems)

These are **very commonly asked in FAANG, product companies, and senior Java developer interviews**.